4b89dcdf-6606-4e87-b667-ecf7c3e8d690

<prev_user>

<request>

You are working on an issue in the 'pavel-kirienko/cavl' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/cavl/cavl (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2025-11-29T22:23:24.358Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section details on the original issue you should resolve

<issue_title>cavl2.h: Add cavl2_lower_bound(), cavl2_predecessor(), and their counterparts</issue_title>
<issue_description></issue_description>

<agent_instructions>Don't forget to ensure test coverage</agent_instructions>

Comments on the Issue (you are @copilot in this section)

<comments>
</comments>

</problem_statement>

Implement the necessary changes to the repository so that the requirements specified in the <problem_statement> are met.

```
  Ensure that you lint, build and test early and iteratively to
  validate each code change thoroughly.
```

<prev_assistant>

```
The implementation is complete. Let me provide a summary of what was
implemented:

## Summary

This PR adds four new functions to `cavl2.h` as requested in the issue:

### New Functions

1. **`cavl2_predecessor(CAVL2_T* node)`**
   - Returns the next smaller node in the in-order traversal of the tree
   - This is the opposite of `cavl2_next_greater()`
   - Enables reverse in-order traversal: `for (CAVL2_T* p =
cavl2_max(root); p != NULL; p = cavl2_predecessor(p))`
   - Time complexity: O(log n) for single call, O(n) for full traversal

2. **`cavl2_successor(CAVL2_T* node)`**
   - An alias for `cavl2_next_greater()` for naming consistency with
`cavl2_predecessor()`
   - Time complexity: O(log n) for single call

3. **`cavl2_lower_bound(CAVL2_T* root, const void* user,
cavl2_comparator_t comparator)`**
   - Finds the smallest node whose value is >= the search target
   - Similar to `std::lower_bound` in C++
   - Returns NULL if no such node exists (all nodes compare less than
target)
   - Time complexity: O(log n)

4. **`cavl2_upper_bound(CAVL2_T* root, const void* user,
cavl2_comparator_t comparator)`**
   - Finds the smallest node whose value is strictly > the search target
   - Similar to `std::upper_bound` in C++
   - Returns NULL if no such node exists (all nodes compare less than or
equal to target)
   - Time complexity: O(log n)
```

Tests Added

Comprehensive tests were added covering:
- Basic functionality of each new function
- Edge cases (NULL inputs, empty trees, boundary values)
- Full traversal verification (predecessor traverses in reverse order)
- Consistency verification (successor == next_greater)
- Lower/upper bound behavior for existing and non-existing values
- Complete boundary testing with 128 even values (0, 2, 4, ... 254) testing all 256 possible search targets

<current>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'pavel-kirienko/cavl' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/cavl/cavl (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2025-11-29T22:54:14.153Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Add cavl2_lower_bound(), cavl2_predecessor(), and their counterparts
</pr_title>

<pr_description>
Adds bound search and reverse traversal functions to cavl2.h.

````
## New Functions

- **`cavl2_predecessor(node)`** - Returns next smaller node in
in-order traversal (inverse of `cavl2_next_greater`)
- **`cavl2_successor(node)`** - Alias for `cavl2_next_greater`
for naming symmetry
- **`cavl2_lower_bound(root, user, comparator)`** - First node ≥
target (like `std::lower_bound`)
- **`cavl2_upper_bound(root, user, comparator)`** - First node
&gt; target (like `std::upper_bound`)

All O(log n) per call.

## Example Usage

```c
// Reverse traversal
for (CAVL2_T* p = cavl2_max(root); p != NULL; p =
cavl2_predecessor(p)) {
    // process nodes in descending order
}

// Range queries
CAVL2_T* first = cavl2_lower_bound(root, &min_key,
comparator);  // first >= min_key
CAVL2_T* end   = cavl2_upper_bound(root, &max_key,
comparator);  // first > max_key
```

## Tests

Comprehensive coverage including edge cases, NULL handling, and
exhaustive boundary testing across 256 search targets against a
tree of 128 even values.

<issue_title>cavl2.h: Add cavl2_lower_bound(),
cavl2_predecessor(), and their counterparts</issue_title>
&gt;<issue_description></issue_description>
&gt;
&gt;<agent_instructions>Don't forget to ensure test coverage</
agent_instructions>
&gt;
&gt; ## Comments on the Issue (you are @copilot in this section)
&gt;
&gt;<comments>
&gt;</comments>
&gt;
````

- Fixes pavel-kirienko/cavl#29
</pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_new>
<comment_id>3520861485</comment_id>
<author>@pavel-kirienko</author>
@copilot remove predecessor and successor, keep only lower/upper bound. Run clang-format and ensure the build is green.
</comment_new>

</pr_comments>

<pr_reviews>

<comment_thread>
<file>c/cavl2.h:240-266</file>
<original_commit>a292480</original_commit>

<comment_new>
<comment_id>2573235530</comment_id>
<author>@pavel-kirienko</author>
```suggestion
/// Find the smallest node whose value is greater than or equal to the search target, in O(log n).
/// Returns the first node for which the comparator returns a non-positive result.
/// If no such node exists (all nodes compare less than target), returns NULL.
/// The comparator function returns: positive if target > candidate, zero if target == candidate, negative if target < candidate.
static inline CAVL2_T* cavl2_lower_bound(CAVL2_T* const          root,
                                         const void* const       user,
                                         const cavl2_comparator_t comparator)
{
    CAVL2_T* result = NULL;
    if ((root != NULL) && (comparator != NULL)) {
        CAVL2_T* n = root;
```

```
        while (n != NULL) {
            const CAVL2_RELATION cmp = comparator(user, n);
            if (cmp <= 0) {
                result = n;
                n      = n->lr[0];
            } else {
                n = n->lr[1];
            }
        }
    }
    return result;
}
```
</comment_new>
</comment_thread>

<comment_thread>
<file>c/cavl2.h:268-295</file>
<original_commit>a292480</original_commit>

<comment_new>
<comment_id>2573236566</comment_id>
<author>@pavel-kirienko</author>
```suggestion
/// Find the smallest node whose value is strictly greater
than the search target (upper bound).
/// Returns the first node for which the comparator returns
a negative result.
/// See cavl2_lower_bound() for details.
static inline CAVL2_T* cavl2_upper_bound(CAVL2_T*
const          root,
                                         const void*
const       user,
                                         const
cavl2_comparator_t comparator)
{
    CAVL2_T* result = NULL;
    if ((root != NULL) && (comparator != NULL)) {
        CAVL2_T* n = root;
        while (n != NULL) {
            const CAVL2_RELATION cmp = comparator(user, n);
            if (cmp < 0) {
                result = n;
                n      = n->lr[0];
            } else {
                n = n->lr[1];
            }
```

```
            }
        }
        return result;
    }

    ```
    </comment_new>
</comment_thread>

<comment_thread>
<file>c/cavl2.h:209-238</file>
<original_commit>a292480</original_commit>
    <comment_new>
    <comment_id>2573236805</comment_id>
    <author>@pavel-kirienko</author>
    ```suggestion
    ```
    </comment_new>
</comment_thread>

</pr_reviews>

</comments>

The last **3** git commits in this branch are the changes you
have made so far. Use those as your change commit history.
</problem_statement>

Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the
<comments> before making any changes. You will only address the
<comment_new> comments. Reply to new comments as you address them.
Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a
suggestion.
    - Is it addressed to you or someone else, e.g. "Hey
@username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can
stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and
```
```

```
ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>


Ensure that you lint, build and test early and iteratively to
validate each code change thoroughly.
```